

Deep SeeK

Not just cheaper to build, also cheaper to run

Open sourced, permissively licensed

DeepSeeK V3 $\rightarrow$ R1

LM: Probabilistic engine for next token

Exponential Rise in Scaling, Lang $\gg$ Vision

Emergent Properties in larger models

- sudden rise after certain point of params / power scaling

GPT was finetuned with Reinforcement Human Feed back Learning

DeepSeeK V3 is 671B params - not as polished or safe as gpt4 / claude

Input $\rightarrow$ Tokenized Text $\rightarrow$ Token Embed $\rightarrow$ Position Embed
(id assign) (are you a -?) (location)

Token + Position = Input Embed (uniform / identity)

Input Emb $\rightarrow$ Norm $\rightarrow$ MHA $\rightarrow$ Dropout $\rightarrow$ Skip $\rightarrow$ Norm $\rightarrow$ FFN
Skip $\leftarrow$ Dropout $\leftarrow$

Skip handles vanishing / exploding gradient problem

Dropout helps avoid lazy params

Input Emb [765] $\rightarrow$ NN $\rightarrow$ Vocab [ndim]

Choose highest prob of next token $\rightarrow$ Actual vs Predicted

$\downarrow$

• RNN has Context Bottleneck

Backpropagate

$\Rightarrow$ Needs Memory

Selectively access parts of input sequence during decoding

⑧

Neural Machine Translation by Jointly Learning to Align and Translate

- Bahdanau Attention

2014: Attention + RNN

2017: Attention + TransSeq

2018: Attention + GPT (decoder only)

Context Vectors

- create enriched representations of each element in an input sequence (like a sentence) by incorporating info from all other elements in the sequence

Higher dot product $\Rightarrow$ Similarity ; can't resolve contextual ambiguities

Cheat: NN figure it out by backpropagation

Query[] and Key[] $\Rightarrow$ QKV resolve ambiguity

$$Q * K^T = \text{Attn.} [] \xrightarrow{\text{softmax}} \text{Attn. wt. } (0-1) \text{ (peaky bad)}$$

$\hookrightarrow$ relation between one to all

$$\downarrow \\ \text{Var} = \sqrt{d_{\text{keys}}}$$

$$\text{Attn wt} * V = \text{Context Vector}$$

Causal Attn / Masked Attn

- cannot look into future

$$\text{Attn Scores} \xrightarrow{\text{softmax}} \text{Attn Wt} \xrightarrow{\text{0's diag}} \text{Masked Attn Scores} \xrightarrow{\text{norm}} \text{Masked Attn Wt}$$

$$\text{Better: Attn. Scores} \xrightarrow{\text{upper triangle Neg. Inf. Mask}} \text{Softmax}$$

MultiHead Attention

Analyze multiple perspectives $\Rightarrow$ Richer understanding

$$\text{head-dim} = \frac{d_{\text{out}}}{n_{\text{heads}}}$$

Use heatviz to visualize each perspective focus

KV Cache

We only need the context vector for last token of input sequence to predict next token

$$\text{Compute Time: NoCache vs Cache ; Size} = l * b * n * h * s * 2 * 2$$

(Quadratic) (Linear)

Multi Query Attention

- Same W_h and W_v values for each head
- Q is diff. for each head
- Reduces perspective by restricting diversity of heads

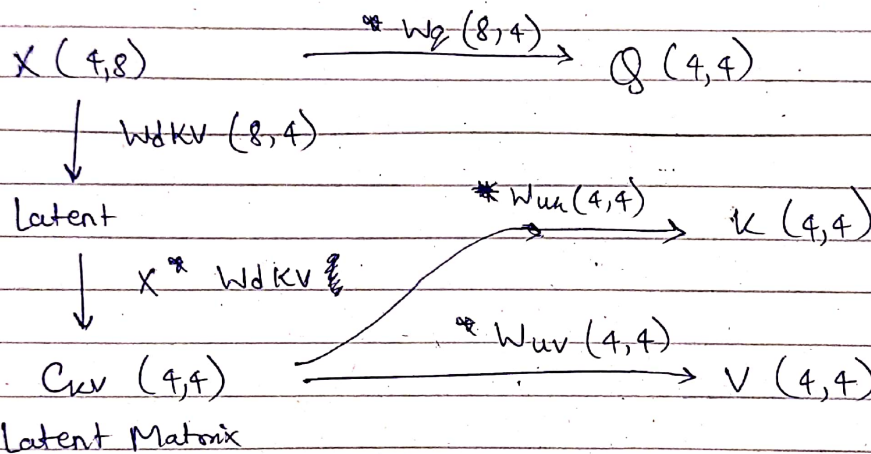
Crossed Query Attention

- Same W_h and W_v values for groups of heads
- Groups perspective
- Higher diversity than MQA with lower KV cache

Multihard Latent Attention

Currently $2^* n^* h$ (K and V matrices)

Take input embeddings X



- Steps:
- 1) $Q = X W_q$
 - 2) $C_{kv} = X W_{dkv}$
 - 3) $K = C_{kv} * W_{hk} = X * W_{dkv} * W_{hk}$
 - 4) $V = C_{kv} * W_{hv} = X * W_{dkv} * W_{hv}$

Absorption Trick: $\text{Attn. Scores} = Q K^T$

$$= X W_q * (W_{hk}^T W_{dkv}^T X^T)$$
$$= X (W_q W_{hk}^T) (X W_{dkv})^T$$

Fixed at training time
one compute only

cached C_{kv}

$$2^* n^* h \rightarrow dl$$

Rotatory Positional Encodings

Without positional info, context vector for same token in diff. positions is same

Token embeds are very small ≈ 0

Position embeds have larger values for contexts // IPE

Net embed often lose tokens ; dilute meaning

Binary PE

LSB oscillate fast between positions ; MSB oscillate slower

Use Binary position info 0's and 1's

During pre-training sudden jumps and discontinuities are difficult for LLMs.

Sinusoidal PE

Continuous values $[-1 \text{ to } 1]$

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{model}})$$

pos depends on context length

Linear Rel

If we have pos encode for position 'p', we must be able to find for position $p+k$

Pos embeds pollute the semantic meaning

- We can augment to Q and K values instead of tokens
- Instead of appending, rotate context vectors, keeping the magnitude

$$Q = W_i p$$

Higher pos $\rightarrow$ larger rotations

Closer queries have similar pos encodes

MHLA + RoPE

$$QK^T \rightarrow R_{pos} (XW_q) * R_{pos} (W_{un}^T W_{dkv}^T X^T)$$

cannot be cached
 R_{pos} is pos dependent token

recomputation required

$$Attn.Sc. = QK^T = [Q_c Q_R] \cdot [K_c K_R]^T$$

$$= [Q_c K_c^T] + [Q_R K_R^T]$$

MHLA

decoupled RoPE

Finally, $KVCache$ and K_R is cached

$$Size = 2 * l * b * s * (d_l + d_{h_R})$$

$4 * h$

$h/2$

MoE

Adaptive Mixtures of Local Experts - Supervised Learning
Feed Forward NN / Expansion-Compaction

Multiple Experts / Sparse Model

- pretrain with far less compute
- faster inference

Crating Network / Router to select expert based on task
ST-MoE

Diss. experts in diss. Transformer blocks

Sparsity : How many experts active for each token ?

Routing : How much weightage for each selected experts ?

$$Input [] * Routing [] = Expert Selector []$$

$\rightarrow$ no. of experts

$\rightarrow$ each col. is an expert

Load Balance $\rightarrow$ Norm
(softmax/var)

Auxiliary Loss Toop

It is added to the main training loss to penalize imbalanced expert selection pushing the routing function toward a more uniform distribution.

Expert Importance of each col.

Balanced MoE Model = Equal importance $\rightarrow$ Penalize variance

$$(\text{coeff. var.}) CV = \text{std. } \sigma / \text{mean } \mu \quad ; \quad \text{Aux. Loss} = \lambda \cdot (CV)^2$$

$\uparrow$
LM Train Loss (next token pred.)

$\rightarrow$ scaling factor

Still not uniform distribution,
Balance Load (Token entropy)

Prob. of choosing (E_i) & frac. of tokens dispatched to each expert

$$\text{Load Balancing Loss} = \text{Scale factor} * \text{No. of experts} * \sum_{i=1}^{\text{noe}} f_i p_i$$

Capacity Factor

$$\text{Expert Capacity} = \text{Tokens per batch} / \text{No. of experts} * CF$$

$$\hookrightarrow \text{batch size} * \text{seq length} * \text{top } k$$

Experts with higher importance (p_i) handle proportionally more tokens thus increase f_i

Auxiliary Loss for Load Balancing

- Aux loss affects Train loss ; λ issues

Avg token load per expert $\Rightarrow$ under or over loaded

$$\text{Load violation error} \Rightarrow b_i = b_i + u * \text{sign}(LVE)$$

Add bias to increase prob of expert choice

Knowledge Hybridity $\rightarrow$ Limited Experts
Knowledge Redundancy

Super Specialized Experts

- Too many experts
- Special + Shared experts

Fine Grained Expert Segmentation

- No. of params and compute costs remain same

① 1 Shared + n Fine Grained Experts

Multi Token Prediction

- Densification of Training Signals
- Improved data efficiency
- Better planning (structure, grammar, forecasting)
- Higher inference speed (self-speculating decoding)

Hidden state $i \rightarrow$ Head $(i+1)$

$\downarrow \downarrow \downarrow$

Shared UnEmbedding (Logits) Matrix $\rightarrow$ Pred. Tokens

Quantization

$$n\text{-bits} = \frac{n}{8} * \text{KB params} \approx \text{GB model}$$

FP16 $\ll$ BF16 (bfloat16)

(half precision)

$\rightarrow$ wider dynamic range ($\approx$ FP32)

$\rightarrow$ 10 bit mantissa

8 bits exponent

better for inference

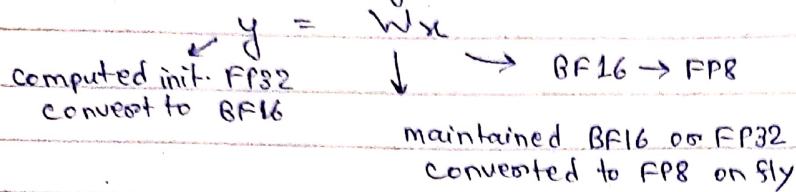
better for training

DeepSeek Quantization

1) Training NN Layer ($y = Wx$)

Input (x): 128×1 ; Weights (W): 256×128 ; Output (y): 256×1

2) Forward Propagation



3) a) Backward Propagation

$$\frac{dL}{dx} = \frac{dL}{dz} * W^T$$

b) Wgrad

$$\frac{dL}{dW} = x^T \frac{dL}{dz}$$

Ideal for GEMM computations

Activation Outputs

Separate (grouped) scaling retains accuracy for second group despite large outliers in first group

Weights

Each chunk scaled separately

Tensor Core and CUDA Core computation

FP8

FP32

Promote to cuda cores for higher precision, WGMMA

4) Precision: Fpass E4M3, Bpass E5M2

DeepSeek: E4M3 for fine grained quantization (no extremes)

5) Online Quantization

calc. scale factor real time (at each Fpass, Bpass) with current Tensor data

VLLM Inference

For user prompt, there are tokens generated $\Rightarrow$ Inference

Inference engine should use GPU efficiently

Open source: vLLM with Paged Attention

Step 0: Arrival "Hello my name is ..."

install vllm; what happens under the hood of `llm.generate(...)`

Tokenizes and Model loaded

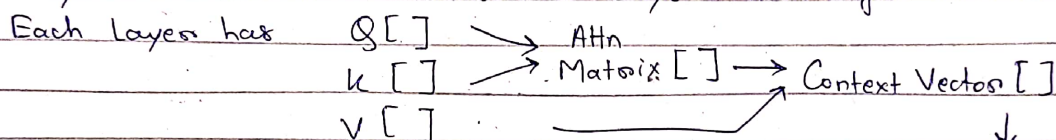
Step 1: Tokenization

All models come with their own, generally BPE

Step 2: Waiting Queue

Prefill Request vs Decoding Request $\uparrow$

TinyLlama 1B has 22 Transformer Layers then logits then next 'Word'



$\downarrow$ NN, Short cut, Normalize

KV Cache reused for next word onwards

Prefill Stage: 1st word predicted without cache

Decoding Stage: 2nd word onwards with cache

For each prompt there is prefill then decoding stages

Priority is towards decoding stages to finish one prompt and KV cache

KV calc. in each layer, stored in 'blocks'.

Rows = Num-Tokens(16); Cols = $K[]$ and $V[]$ ($n_{\text{heads}} * \text{head-dim}$)

Each transformer layer has n blocks

$P_i \rightarrow B_i$

CPU VRAM has mostly KV Cache + Model Weights +

Pytorch Activation peak memory + Non torch memory

(0.3GB)

(0.05GB)

Each block memory taken = $2 * \text{Num-Tokens} * n_{\text{head}} * \text{head-dim} * \text{bytes per param}$

(kv) (16)

(FP16/INT8)

Sample

1) Model dims (from HF config)

- num-hidden-layers = 22
- num-attn-heads = 32
- num-key-value-heads = 4 // paged attention
- hidden-size = 2048 $\rightarrow$ head-dim = $2048/32 = 64$

2) Bytes per token of KV cache

$$\begin{aligned} \text{bytes-per-token} &= 2(k, v) * n\text{-layers} * n\text{heads} * \text{head-dim} \\ &\quad * \text{bytes-per-element (FP16)} \\ &= 2 * 22 * 4 * 64 * 2 = 22528 \text{ bytes/token} = 22 \text{ KiB/token} \end{aligned}$$

3) Bytes per block

vLLM default block size = 16 tokens $\rightarrow$

$$\text{bytes-per-block} = 22 \text{ KiB/token} * 16 = 0.34 \text{ MiB/block}$$

4) # CUDA blocks (GPU, pool)

Budget from KV cache = 10 GiB

$$\Rightarrow 10 / 0.34 \approx 32000 \text{ blocks}$$

CPU blocks (host swap pool)

default = 4 GiB per GPU in vLLM

$$\Rightarrow 4 / 0.34 \approx 12000 \text{ blocks}$$

Step 3: Prefill Phase

At this point none of the ~~these~~ requests have been processed so they are all in waiting queue (needing prompt processing)

There are no running requests yet (no one is in decoding, since no prompt processed)

The scheduler examines the token budget (2048 tokens), and compares to total tokens of all input prompts hence all done in same iteration

↓

Continuous Batching

- Batch of input tokens []
- Positions []

↓

Paged KV Cache

CPU maintains Page Table and Free Block Queue for each layer
(alloc. block) (empty block)

Step 4: Decoding Phase

Scheduler moves all requests into running queue

KV cache of prev tokens + KV of new token → Logits → Next token

When a prompt reaches EOS token, it is cleared and added to Free Block Queue

VLM allocates next prompt to these free blocks.

In Continuous Batching, when a sequence finishes, that block is freed and immediately starts generating next sequence.

No idle memory ⇒ Efficient usage ⇒ Paged Attention